

Operators in Python

Understanding the Building Blocks of Python
Expressions

Presented by: Asif Farooq

What are Operators?

- ▶ Operators are special symbols in Python used to perform operations on variables and values.
- ▶ Python supports a variety of operators to perform different types of tasks.

Types of Operators in Python

1. Arithmetic Operators
2. Comparison (Relational) Operators
3. Assignment Operators
4. Logical Operators
5. Bitwise Operators
6. Membership Operators
7. Identity Operators

Arithmetic Operators

- ▶ `+` : Addition $\rightarrow 3 + 2 = 5$
- ▶ `-` : Subtraction $\rightarrow 5 - 1 = 4$
- ▶ `*` : Multiplication $\rightarrow 4 * 3 = 12$
- ▶ `/` : Division $\rightarrow 10 / 2 = 5.0$
- ▶ `//` : Floor Division $\rightarrow 7 // 2 = 3$
- ▶ `%` : Modulus $\rightarrow 7 \% 2 = 1$
- ▶ `**` : Exponentiation $\rightarrow 2 ** 3 = 8$

Comparison Operators

- ▶ `==` : Equal to $\rightarrow 5 == 5 \rightarrow \text{True}$
- ▶ `!=` : Not equal to $\rightarrow 5 != 3 \rightarrow \text{True}$
- ▶ `>` : Greater than $\rightarrow 5 > 3 \rightarrow \text{True}$
- ▶ `<` : Less than $\rightarrow 2 < 4 \rightarrow \text{True}$
- ▶ `>=` : Greater than or equal $\rightarrow 5 >= 5 \rightarrow \text{True}$
- ▶ `<=` : Less than or equal $\rightarrow 3 <= 4 \rightarrow \text{True}$

Assignment Operators

- ▶ `=` : $x = 5$
- ▶ `+=` : $x += 3 \rightarrow x = x + 3$
- ▶ `-=` : $x -= 2 \rightarrow x = x - 2$
- ▶ `*=` : $x *= 4 \rightarrow x = x * 4$
- ▶ `/=` : $x /= 2 \rightarrow x = x / 2$
- ▶ `//=` : $x //= 2 \rightarrow x = x // 2$
- ▶ `%=` : $x \% = 3 \rightarrow x = x \% 3$
- ▶ `**=` : $x ** = 2 \rightarrow x = x ** 2$

Logical Operators

- ▶ and : True if both statements are true $\rightarrow x > 3$
and $x < 10$
- ▶ or : True if at least one is true $\rightarrow x < 3$ or $x > 10$
- ▶ not : Reverses the result $\rightarrow \text{not}(x > 5)$

Bitwise Operators

- ▶ `&` : AND $\rightarrow 5 \& 3 = 1$
- ▶ `|` : OR $\rightarrow 5 | 3 = 7$
- ▶ `^` : XOR $\rightarrow 5 ^ 3 = 6$
- ▶ `~` : NOT $\rightarrow \sim 5 = -6$
- ▶ `<<` : Left Shift $\rightarrow 5 << 1 = 10$
- ▶ `>>` : Right Shift $\rightarrow 5 >> 1 = 2$

Membership Operators

- ▶ `in` : True if value is in sequence → 'a' in 'apple' → True
- ▶ `not in` : True if value is not in sequence → 'x' not in 'apple' → True

Identity Operators

- ▶ `is` : True if both refer to same object → `x is y`
- ▶ `is not` : True if they do not refer to same obj → `x is not y`

Operator Precedence

- Order of operations matters

- Example:

$2 + 3 * 4 \rightarrow 14$ (Multiplication first)

- Use parentheses to control precedence:

$(2 + 3) * 4 \rightarrow 20$

Summary

- Operators make expressions and logic possible in Python.
- Mastering operators is essential for writing efficient code.